# System Requirements Specification - Anti-Front-Running ZK Perpetuals on Cardano using Midnight

Version: **v0.1.0**

Date: **March 2026**

License: **MIT**

---

# 1. Introduction

## 1.1 Purpose

This document specifies the system requirements for a **privacy-preserving perpetual derivatives trading protocol built using Cardano and the Midnight network**.

The system introduces **private order flow and verifiable settlement using zero-knowledge proofs**, preventing front-running and MEV exploitation in decentralized derivatives markets.

The protocol leverages:

- Midnight confidential smart contracts
- zero-knowledge proof verification
- selective disclosure mechanisms
- private state transitions verified through cryptographic proofs

Zero-knowledge proofs allow a participant to prove the correctness of a statement **without revealing the underlying private data**, enabling confidential financial operations on a public blockchain.

---

# 2. System Overview

## 2.1 Problem Statement

On most public blockchains, transactions are visible in the mempool before confirmation.

This transparency allows adversaries to:

- observe pending trades
- insert competing transactions
- reorder execution
- extract profit through MEV strategies

These attacks cause:

- unfair execution
- strategy leakage
- slippage manipulation
- loss of capital for legitimate traders

Perpetual derivatives trading is particularly vulnerable because **execution order and timing directly influence profitability**.

---

## 2.2 Solution Overview

The system introduces a **confidential trading infrastructure using Midnight's privacy framework**.

Key principles:

1. **Private transaction execution**
2. **Selective disclosure of data**
3. **ZK-verified state transitions**
4. **Proof-based settlement verification**

Midnight enables applications to prove transaction validity **without exposing sensitive data**, providing programmable privacy for decentralized applications.

---

# 3. Functional Requirements

## 3.1 Private Order Submission

The system must allow traders to submit perpetual trading orders including:

- trading pair
- order direction (long / short)
- leverage
- margin allocation
- price
- order size

Order parameters must remain **confidential and protected by cryptographic commitments**.

---

## 3.2 Order Commitment

Orders must be represented as **commitments generated locally by the trader**.

Each commitment must contain:

- hashed order parameters
- trader signature
- nonce value

This prevents adversaries from inspecting trade parameters prior to execution.

---

## 3.3 Proof-Based Order Validation

Before an order can be processed, the system must verify a **zero-knowledge proof demonstrating**:

- the trader owns sufficient collateral
- the order format is valid
- margin requirements are satisfied

The verification must occur **without revealing the private values used in the proof**.

---

## 3.4 Confidential Trade Matching

Orders are matched according to protocol rules:

- price compatibility
- liquidity availability
- margin constraints

Matching logic may occur:

- within private execution environments
- through cryptographic verification circuits

Correct matching must be verified through proof generation before settlement.

---

## 3.5 Settlement

Once a match occurs, the system must update:

- trader balances
- margin accounts
- open positions

State transitions must be verified using **zero-knowledge proofs before being accepted by the network**.

---

## 3.6 Liquidation Engine

The protocol must support liquidation when margin requirements are violated.

Liquidation must verify:

- margin ratios
- position exposure
- liquidation penalties

The liquidation decision must be validated using a cryptographic proof to prevent manipulation.

---

# 4. Non-Functional Requirements

## 4.1 Privacy

The protocol must protect the confidentiality of:

- order parameters
- trader identities
- margin balances
- open positions

---

## 4.2 Security

The protocol must prevent:

- front-running
- transaction replay attacks
- order tampering
- malicious validator manipulation

---

## 4.3 Scalability

The architecture must support high-frequency trading workloads without significant latency increases.

---

## 4.4 Reliability

The system must ensure correct settlement even during high network usage or adversarial conditions.

---

# 5. System Architecture

## 5.1 High Level Architecture

```
Trader Interface
       |
Local Order Construction
       |
Order Commitment
       |
Private Execution Layer
       |
Proof Generation
       |
Midnight Smart Contract Verification
       |
Cardano Anchored Settlement
```

---

## 5.2 Core System Modules

### zk-circuits

Responsible for generating and verifying cryptographic proofs for:

- order validation
- matching correctness
- settlement updates
- liquidation conditions

---

### matching

Handles:

- order intake
- matching logic
- liquidity coordination

---

### settlement

Handles:

- margin updates
- position management
- liquidation execution

---

### privacy

Implements:

- encryption
- commitment generation
- identity shielding

---

### common

Contains shared components including:

- cryptographic utilities
- protocol configuration
- data structures

---

# 6. Midnight Integration

Midnight provides the confidential execution environment for the protocol. The architecture leverages three main Midnight components.

#### 6.1 Kachina Execution Framework

The protocol uses the Kachina framework, which allows users to maintain private state locally and submit zero-knowledge proofs demonstrating valid state transitions. Only the proof is submitted to the blockchain, ensuring sensitive data remains private.

### 6.2 Compact Smart Contract Language

Smart contracts are written using Compact, Midnight's smart contract language designed with privacy as the default behavior. Data must be explicitly disclosed, preventing accidental leakage of sensitive information.

### 6.3 Proof Server

Proof generation is performed using a local proof server, which generates cryptographic proofs validating the correctness of private computations before submission to the network.

# 7. Zero-Knowledge Proof Design

The system relies on **zk-SNARKs using Midnight's Halo 2 implementation**, a PLONK-based proving system optimized for flexible circuit design and recursion.

The Compact compiler generates circuits automatically from contract logic; however, **engineering estimates for circuit complexity are provided below to satisfy design requirements and guide performance expectations**.

---

## 7.1 Circuit Overview

The protocol consists of four primary circuits:

---

### 1. Order Validation Circuit

**Purpose:**

- Validate trader signature
- Ensure sufficient collateral
- Verify order structure integrity

**Estimated Complexity:**

- Constraints: **25,000 – 45,000**
- Advice columns: **8 – 12**
- Lookup tables: **2 – 4**
- Proof generation time: **~1 – 3 seconds (desktop CPU)**

**Notes:**

Signature verification (e.g., EdDSA) dominates constraint count.

---

## 2. Matching Verification Circuit

**Purpose:**

- Validate correct order pairing
- Enforce price compatibility
- Ensure fair execution rules

**Estimated Complexity:**

- Constraints: **40,000 – 80,000**
- Advice columns: **10 – 16**
- Lookup tables: **3 – 6**
- Proof generation time: **~2 – 5 seconds**

**Notes:**

Complexity increases with matching logic and conditional constraints.

---

## 3. Settlement Verification Circuit

**Purpose:**

- Verify balance updates
- Validate position changes
- Enforce margin accounting rules

**Estimated Complexity:**

- Constraints: **60,000 – 120,000**

- Advice columns: **12 – 20**
- Lookup tables: **4 – 8**
- Proof generation time: **~3 – 8 seconds**

**Notes:**

Arithmetic-heavy circuit due to financial state transitions.

---

### 4. Liquidation Verification Circuit

**Purpose:**

- Detect margin breaches
- Validate liquidation eligibility
- Apply penalties correctly

**Estimated Complexity:**

- Constraints: **30,000 – 70,000**
- Advice columns: **8 – 14**
- Lookup tables: **3 – 5**
- Proof generation time: **~2 – 4 seconds**

---

## 7.2 Aggregation Strategy

To improve scalability:

- Multiple proofs may be **aggregated into a single recursive proof**
- Batch settlement verification may reduce on-chain verification load

**Projected aggregated proof complexity:**

- Constraints: **~150,000 – 300,000 (recursive circuit)**
- Verification cost: **constant-size proof (Halo 2)**

---

## 7.3 Design Assumptions

- Hardware baseline: consumer-grade CPU (8-core)
- No GPU acceleration assumed (conservative estimate)
- Circuits compiled via Midnight Compact toolchain
- Constraint ranges are **engineering estimates based on comparable Halo 2 implementations**

# 8. Performance Benchmarks

The following performance benchmarks define target system capabilities, calibrated against observed Midnight network behavior and the characteristics of Halo 2-based proof systems.

| Metric | Target | Rationale |
|---|---|---|
| Proof generation time (per circuit) | 10–60 seconds | Midnight transactions currently exhibit 20–60 second delays attributable to local proof generation via the proof server. Simpler circuits (5K–10K constraints) target the lower end; complex circuits (15K–20K constraints such as settlement) target the upper end. GPU-accelerated proof servers (e.g., CUDA-based implementations) can reduce these times further. |
| Proof verification time (on-chain) | ≤ 10 milliseconds | zk-SNARK verification in Halo 2 is a constant-time operation independent of circuit size, involving a small number of elliptic curve operations and pairing checks. PLONK proof sizes are approximately 400 bytes, and verification completes in single-digit milliseconds. The 10 ms target includes Impact VM overhead for processing the proof within a Midnight transaction. |
| Settlement throughput | ≥ 5 trades per second | Constrained by Midnight block production cadence and the sequential nature of ledger state updates per contract. Batch proof aggregation and parallelized proof generation across independent trading pairs can improve effective throughput. |
| Order commitment | ≤ 2 seconds | Commitment generation is a local operation (hash computation + nonce generation) that does not |

| Metric | Target | Rationale |
|---|---|---|
| latency | | require proof generation. This target reflects client-side computation only. |
| Proof generation time (GPU-accelerated) | 5–20 seconds | Target for deployments using GPU-accelerated proof servers. Community-built CUDA-accelerated Midnight proof servers have demonstrated significant speedups over CPU-only proving. |

> **Note:** Proof generation is the computationally intensive step and occurs locally on the user's machine via Midnight's proof server (typically a Docker container at localhost:6300). The proof server requires access to private inputs by design — using a remote proof server would compromise user privacy. Contracts with higher constraint counts require proportionally longer proof generation times and more computational resources. All targets are engineering estimates that will be validated during development against Midnight's proof server performance on reference hardware (8-core CPU, 16 GB RAM baseline; NVIDIA RTX-class GPU for accelerated targets). On-chain verification is lightweight and constant-time regardless of circuit complexity, consistent with PLONK/Halo 2 verification characteristics.

# 9. Privacy Guarantees

The protocol guarantees the following privacy properties.

## 9.1 Order Confidentiality

Trade parameters remain encrypted and are never revealed to external observers.

## 9.2 Participant Anonymity

Trader identities remain protected through cryptographic commitments and shielded execution.

## 9.3 Strategy Protection

Observers cannot infer trading strategies from:

- order flow
- settlement transactions
- state updates

---

# 10. Threat Model

## 10.1 Adversaries

Potential adversaries include:

- MEV bots
- malicious traders
- compromised nodes
- external attackers

---

## 10.2 Cryptographic Assumptions

Security relies on:

- correctness of zero-knowledge proof verification
- collision resistance of hash functions
- secure digital signature schemes
- integrity of Midnight's execution framework

---

# 11. Attack Vectors and Mitigations

| Attack | Description | Mitigation |
|---|---|---|
| Front-running | Bots inspect pending transactions | Private order commitments |
| Replay attacks | Reusing previously signed orders | Nonce verification |
| Order manipulation | Tampering with order parameters | ZK proof validation |

| Attack | Description | Mitigation |
|---|---|---|
| Validator collusion | Manipulating execution order | Confidential execution |
| State forgery | Invalid settlement state | Proof-verified transitions |

# 12. Risk Assessment

## Technical Risks

| Risk | Impact |
|---|---|
| inefficient proof generation | increased latency |
| circuit complexity | reduced scalability |
| cryptographic vulnerabilities | protocol compromise |

## Mitigation Strategies

- incremental circuit optimization
- batch proof aggregation
- formal security audits
- staged testnet deployment

# 13. Repository Structure

The public repository will contain the following modules:

```
root
│
├── zk-circuits
├── matching
├── settlement
├── privacy
├── common
```

```
└── docs
      └── SRS.md
```